Program Design Quiz 6

(Please use C language to solve the problem)

題目敘述：

在數論中，若有整數 a,b,c滿足 $a=b\times c$，則稱 a 是 b 或 c 的倍數(multiple)，而 b 或 c 則稱為 a 的除數或因數（divisor / factor）。
若 $c\neq\pm1$，則稱 b 為 a 的「真除數」(proper divisor)。

偶數（even integer）是 2 的倍數（包含 0），例如：-4、0、2、10。
奇數（odd integer）則是「不是偶數」的整數，例如：-5、1、3、9。

完全數（perfect number） 指一個「正整數」等於其所有「正的真除數」總和的數。
例如：

- 6 的正真除數為 1、2、3，有 $6=1+2+3$，所以 6 是完全數。
- 28 的正真除數為 1、2、4、7、14，有 $28=1+2+4+7+14$，所以 28 也是完全數。

不是完全數的正整數稱為 **非完全數（imperfect）**。
對於非完全數，又可依「正真除數總和」與該數本身大小的關係，再分為：

- **虧數（deficient）**：所有正真除數的總和 **小於** 該數本身。
- **過剩數（abundant）**：所有正真除數的總和 **大於** 該數本身。

例如：

- 9 的正真除數為 1、3，總和為 4，小於 9，所以 9 是虧數(DEFICIENT)。
- 12 的正真除數為 1、2、3、4、6，總和為 16，大於 12，所以 12 是過剩數（ABUNDANT）。

因此，請你用 C 語言，給定一個正整數，判斷它是 **PERFECT**（完全數）、**DEFICIENT**（虧數）或 **ABUNDANT**（過剩數）。

輸入說明：

輸入為一串正整數，每個皆不大於 60000。設其個數為 N，滿足 $1<N<100$。
整個序列的最後會有一個整數 0，代表輸入結束，0 本身不需進行判斷。

輸出說明：

輸出的第一行必須為：
PERFECTION OUTPUT
接著對於每一個輸入的正整數（不包含結尾的 0），各輸出一行結果。
每一行的格式如下：

- 先輸出該整數本身，**在寬度為 5 的欄位中右對齊**（右靠齊，左側以空白

補齊)。

- 接著輸出 **兩個空白字元**。
- 然後輸出該整數的判別結果字串：PERFECT、DEFICIENT 或 ABUNDANT 之一。

最後一行輸出：
END OF OUTPUT

Examples
==========
Input 1:
15 28 6 56 60000 22 496 0

Output 1:
PERFECTION OUTPUT
```
    15  DEFICIENT
    28  PERFECT
     6  PERFECT
    56  ABUNDANT
 60000  ABUNDANT
    22  DEFICIENT
   496  PERFECT
```
END OF OUTPUT

Program Design Quiz 5

(Please use C language to solve the problem)

Problem Description：

In number theory, if there exist integers a, b, c such that

$$a = b \times c,$$

then a is called a multiple of b (or c), and b (or c) is called a divisor (or factor) of a.

If, in addition, $c \neq \pm 1$, then $b$is called a *proper divisor* of a.

An *even integer* is any multiple of 2 (including 0), for example: $-4,0,2,10$.

An *odd integer* is an integer that is not even, for example: $-5,1,3,9$.

A **perfect number** is a positive integer that is equal to the sum of all its positive proper divisors.

For example:

- The positive proper divisors of 6 are 1, 2, and 3, and $6 = 1 + 2 + 3$, so 6 is a perfect number.
- The positive proper divisors of 28 are 1, 2, 4, 7, and 14, and $28 = 1 + 2 + 4 + 7 + 14$, so 28 is also a perfect number.

A positive integer that is not perfect is called **imperfect**.

Imperfect numbers can be further classified according to the sum of their positive proper divisors:

- **Deficient number**: the sum of its positive proper divisors is **less than** the number itself.
- **Abundant number**: the sum of its positive proper divisors is **greater than** the number itself.

For example:

- The positive proper divisors of 9 are 1 and 3; their sum is 4, which is less than 9, so 9 is **DEFICIENT**.
- The positive proper divisors of 12 are 1, 2, 3, 4, and 6; their sum is 16, which is greater than 12, so 12 is **ABUNDANT**.

Input：

The input consists of a sequence of positive integers, each not greater than 60000.

Suppose there are Nsuch integers, where $1 < N < 100$.

The sequence is terminated by a single integer 0.

The terminating 0 is **not** to be processed.

Output：

The first line of the output must be:

PERFECTION OUTPUT

For each positive integer in the input (excluding the final 0), output one line with the following format:

- First, print the integer itself, **right-justified in a field of width 5**.
- Then print **two spaces**.
- Then print one of the strings: PERFECT, DEFICIENT, or ABUNDANT.

After all results have been printed, output the final line:

END OF OUTPUT

Examples

==========

Input 1:

15 28 6 56 60000 22 496 0

Output 1:

```
PERFECTION OUTPUT
   15  DEFICIENT
   28  PERFECT
    6  PERFECT
   56  ABUNDANT
60000  ABUNDANT
   22  DEFICIENT
  496  PERFECT
END OF OUTPUT
```